MD AAFI SHAHID MANSURI 002073074
TERRY ZHEN 001859641

CS 5330: Pattern Recognition & Computer Vision

Project 5: Recognition using Deep Networks

Aafi Mansuri, Terry Zhen

Project Overview:

This project explores building, training, and analyzing deep networks for image recognition using the MNIST digit dataset. We start by implementing a convolutional neural network (CNN) in PyTorch to classify handwritten digits, then examine the network's internal structure by visualizing its learned filters and their effects on input images. The trained network is tested on both the standard test set and our own handwritten digits. We then apply transfer learning to adapt the pretrained MNIST network for recognizing Greek letters (alpha, beta, gamma) using only 27 training examples. Next, we replace the CNN architecture with a Vision Transformer that processes images as sequences of patches rather than using convolution filters. Finally, we design and run an experiment to evaluate how different network parameters affect classification performance.

Contributions:

Terry: MNIST network architecture, training and evaluation, test network predictions, handwritten digit testing, network examination and filter visualization, experiment design and evaluation

Aafi: Greek letter transfer learning, Vision Transformer implementation, extended Greek letter classification (6 classes), interactive Gradio digit recognition app

Required Images:

Task 1: Build and train a network to recognize digits

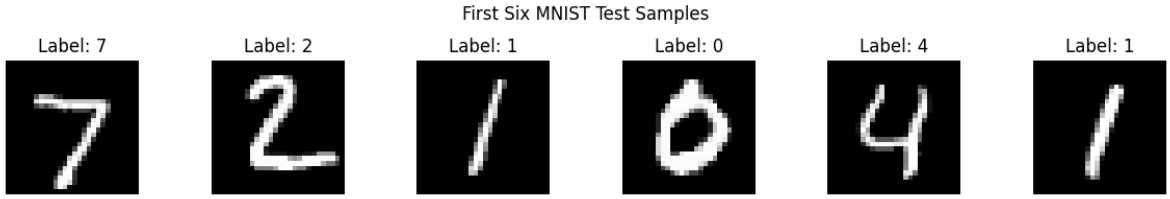
For task1, we built, trained, and saved a convolutional neural network that can recognize handwritten digits from the MNIST datasets. The network follows the requested architecture from the assignment which can be seen in the diagram for part B.

The first block applies 10 5x5 filters, then max-pools and applies ReLU. The second block applies 20 5x5 filters and passes through a dropout layer with p=0.5 for regularization then again max-pools and applies ReLU. The resulting feature maps are flatened into 320-dimensional vector, passed through a fully connected layer with 50 nodes and ReLU, and then finally through an output layer with 10 nodes and log-softmax where each node represents a digit class.

As for training, we used the full 60k MNIST training set with SGD w/ learning rate of 0.01 and momentum of 0.5 and a default batch size of 64 which is configurable from the command line. The model was trained for 5 epochs by default, which is also configurable from the command line and evaluates loss and accuracy on both the training and test set after each epoch.

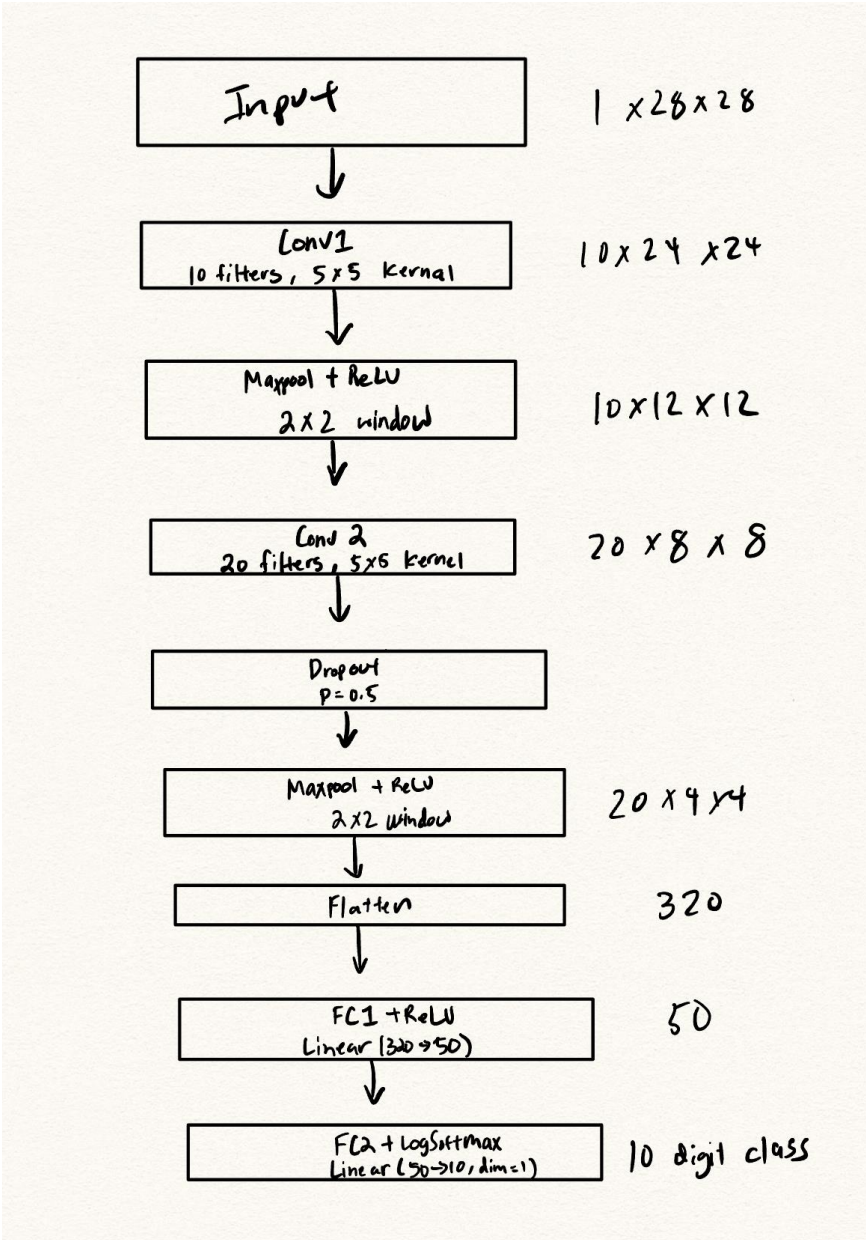
We also tested our model using the first ten samples of our MNIST test set as well as 10 handwritten digits that the model hasn't been seen yet. For the handwritten digits, we had to preprocess each image, so they matched the MNIST format. We converted the original images to grayscale, resized them to 28x28 using Lancoz downsampling, and inverted the digits to appear white on black, and we applied the same normalization we used for the training set. We were able to classify all samples correctly using our model and the results can be seen in the images below.

Part A:

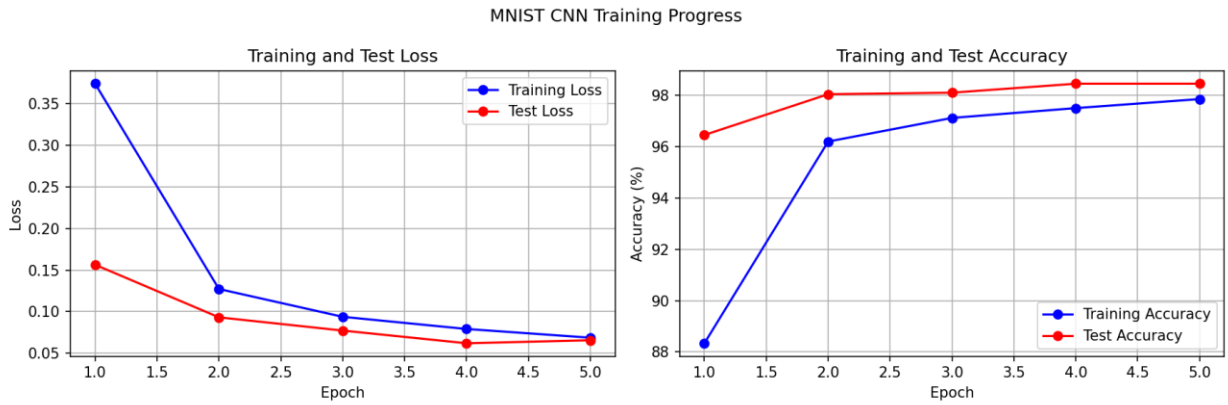


Part B (Architecture Diagram):

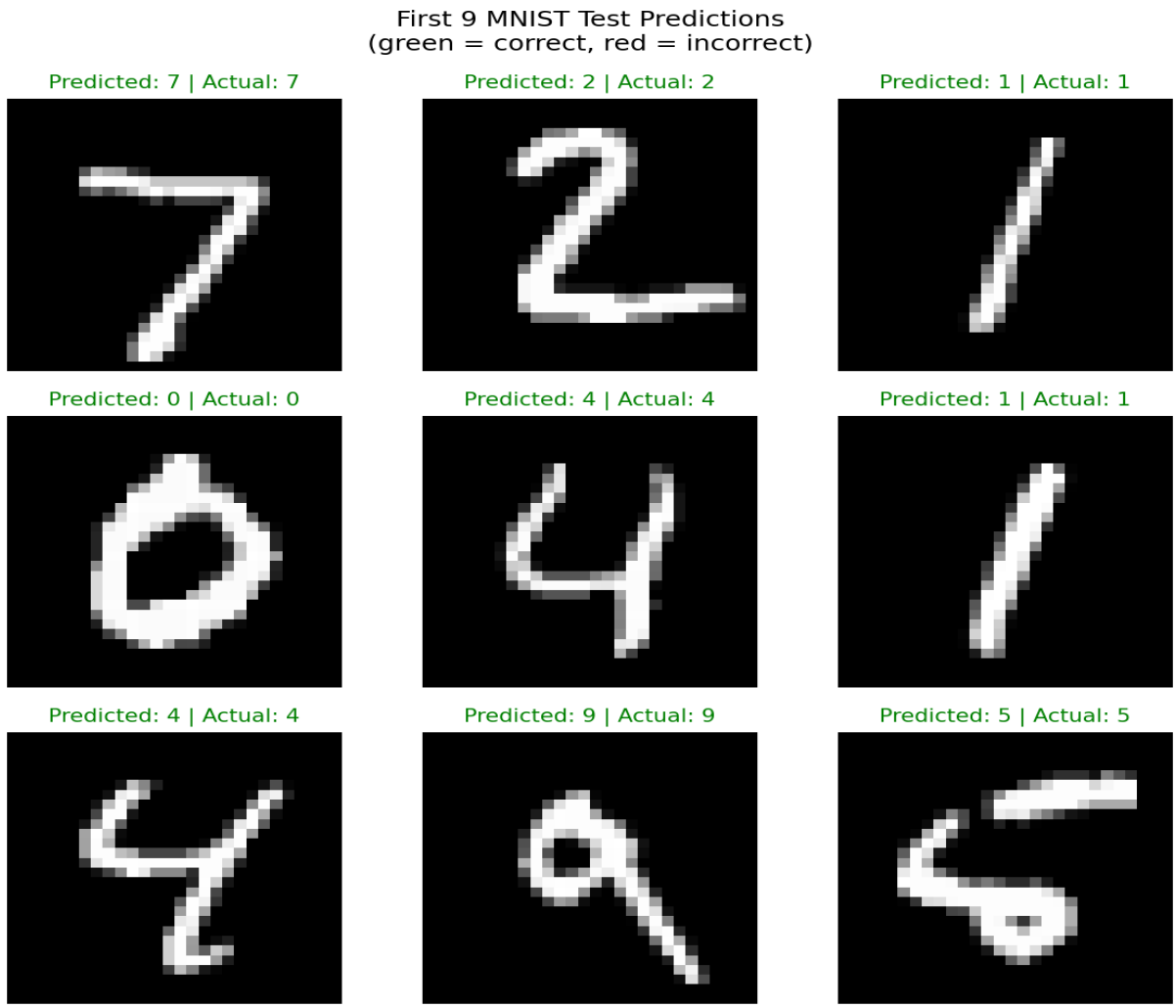
MD AAFI SHAHID MANSURI 002073074
TERRY ZHEN 001859641



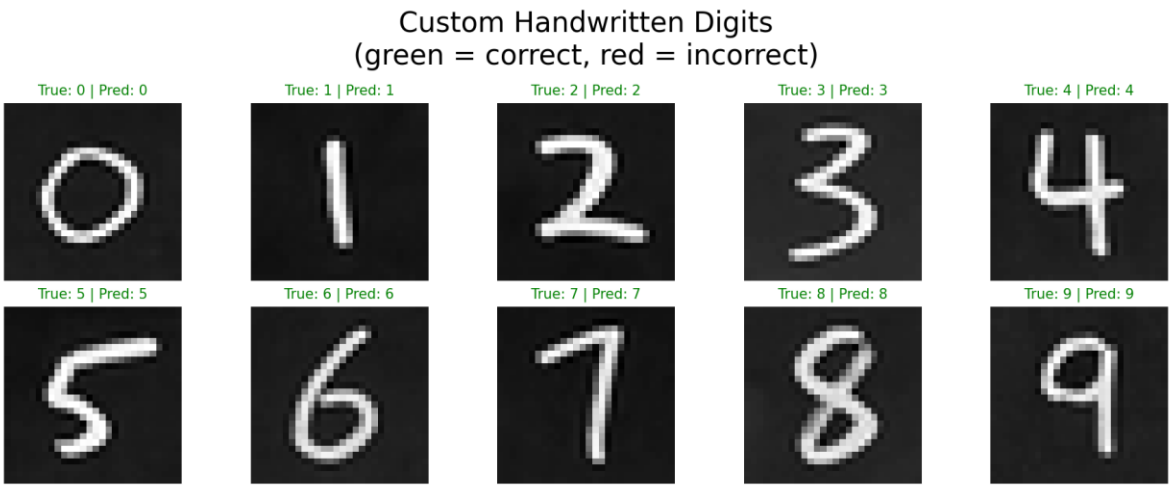
Part C:


MNIST CNN Training Progress
Training and Test Loss
Training Loss
Test Loss
Loss
Epoch
Training and Test Accuracy
Training Accuracy
Test Accuracy
Accuracy (%)
Epoch

Part E:


First 9 MNIST Test Predictions
(green = correct, red = incorrect)
Predicted: 7 | Actual: 7
Predicted: 2 | Actual: 2
Predicted: 1 | Actual: 1
Predicted: 0 | Actual: 0
Predicted: 4 | Actual: 4
Predicted: 1 | Actual: 1
Predicted: 4 | Actual: 4
Predicted: 9 | Actual: 9
Predicted: 5 | Actual: 5

Part F:



Task 2: Examine the network

For task 2, we loaded our trained model and analyzed how the first convolutional layer processed data and applied OpenCV's filter2D to apply the 10 filters to the first MNIST training image.

Model Structure Printout:

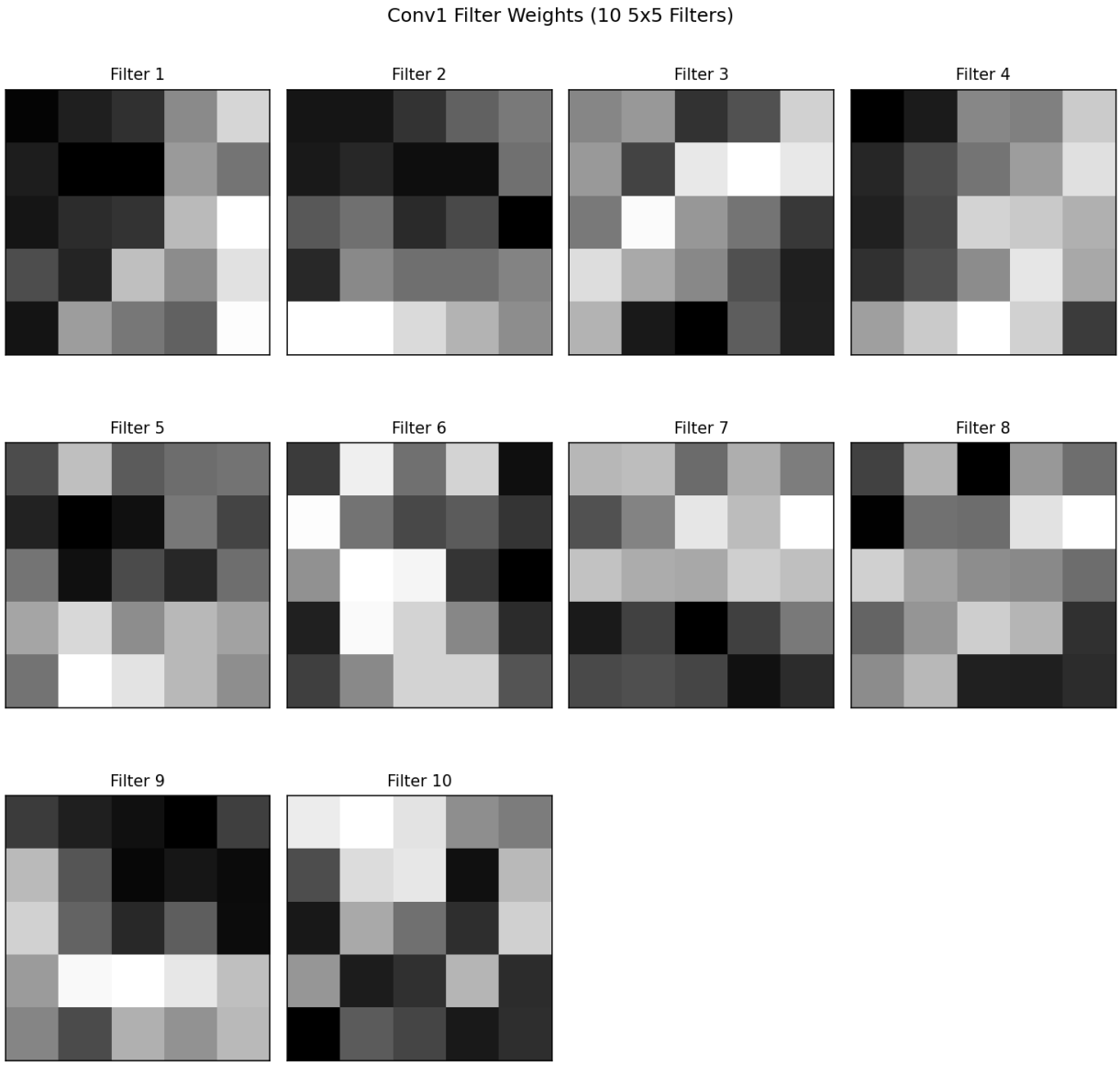
```
Model Structure:
MyNetwork(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1))
  (pool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2): Conv2d(10, 20, kernel_size=(5, 5), stride=(1, 1))
  (dropout): Dropout(p=0.5, inplace=False)
  (pool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (fc1): Linear(in_features=320, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
)
```

Part A:

```
Conv1 weight shape: torch.Size([10, 1, 5, 5])

Filter 1:
Shape: torch.Size([5, 5])
tensor([[-0.2295, -0.1708, -0.1312,  0.0618,  0.2267],
        [-0.1743, -0.2388, -0.2393,  0.0959,  0.0130],
        [-0.1923, -0.1427, -0.1267,  0.1644,  0.3158],
        [-0.0709, -0.1595,  0.1765,  0.0646,  0.2502],
        [-0.1953,  0.1026,  0.0195, -0.0283,  0.3103]],
       grad_fn=<SelectBackward0>)

Filter 2:
Shape: torch.Size([5, 5])
tensor([[-0.2120, -0.2124, -0.1469, -0.0451,  0.0098],
        [-0.2032, -0.1724, -0.2294, -0.2284, -0.0118],
        [-0.0645, -0.0110, -0.1664, -0.0966, -0.2608],
        [-0.1682,  0.0451, -0.0136, -0.0135,  0.0321],
        [ 0.3078,  0.3062,  0.2248,  0.1396,  0.0544]],
       grad_fn=<SelectBackward0>)

Filter 3:
Shape: torch.Size([5, 5])
tensor([[ 0.0274,  0.0724, -0.1890, -0.1068,  0.2176],
        [ 0.0752, -0.1455,  0.2762,  0.3371,  0.2778],
        [-0.0040,  0.3257,  0.0692, -0.0205, -0.1728],
        [ 0.2479,  0.1171,  0.0315, -0.1111, -0.2365],
        [ 0.1422, -0.2526, -0.3175, -0.0786, -0.2309]],
       grad_fn=<SelectBackward0>)
```

```
Filter 4:
Shape: torch.Size([5, 5])
tensor([[-0.2300, -0.1688,  0.0731,  0.0569,  0.2254],
        [-0.1440, -0.0534,  0.0313,  0.1212,  0.2728],
        [-0.1571, -0.0683,  0.2454,  0.2200,  0.1640],
        [-0.1188, -0.0457,  0.0836,  0.2849,  0.1457],
        [ 0.1264,  0.2236,  0.3423,  0.2375, -0.0981]],
       grad_fn=<SelectBackward0>)

Filter 5:
Shape: torch.Size([5, 5])
tensor([[-9.6411e-02,  1.5655e-01, -6.3540e-02, -2.3675e-02, -1.0085e-02],
        [-1.8920e-01, -2.6448e-01, -2.2887e-01,  1.3299e-04, -1.1464e-01],
        [-9.5829e-03, -2.2843e-01, -9.7984e-02, -1.7785e-01, -2.1851e-02],
        [ 9.9601e-02,  2.1012e-01,  4.5248e-02,  1.3991e-01,  9.1567e-02],
        [-1.1037e-02,  2.9778e-01,  2.3752e-01,  1.4018e-01,  4.8352e-02]],
       grad_fn=<SelectBackward0>)

Filter 6:
Shape: torch.Size([5, 5])
tensor([[-0.1431,  0.2077, -0.0407,  0.1538, -0.2291],
        [ 0.2338, -0.0331, -0.1169, -0.0803, -0.1576],
        [ 0.0238,  0.2397,  0.2192, -0.1538, -0.2589],
        [-0.1964,  0.2298,  0.1524,  0.0054, -0.1737],
        [-0.1346,  0.0090,  0.1548,  0.1542, -0.0940]],
       grad_fn=<SelectBackward0>)
```

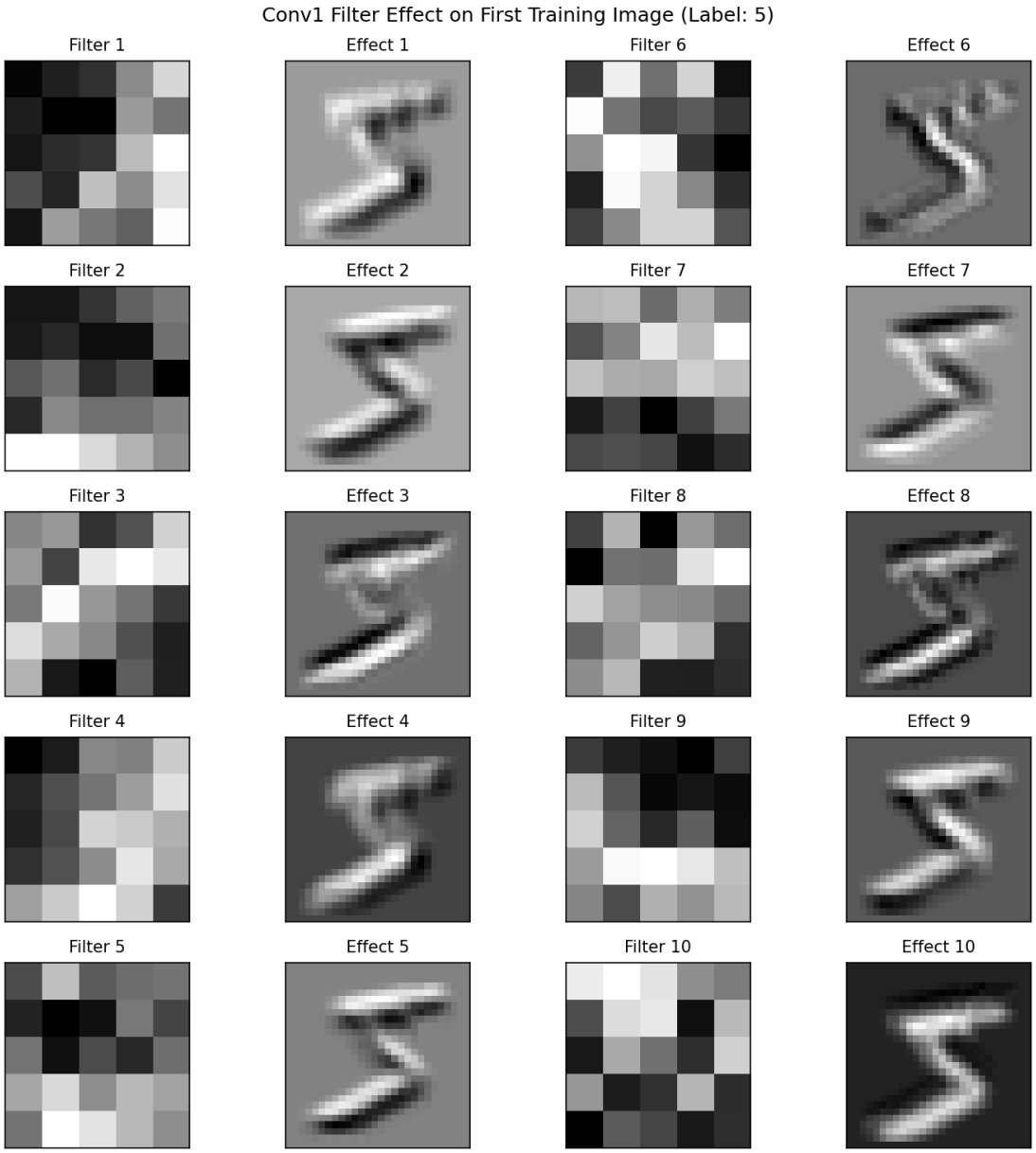
```
Filter 7:
Shape: torch.Size([5, 5])
tensor([[ 0.1455,  0.1632, -0.0740,  0.1187, -0.0218],
        [-0.1492, -0.0053,  0.2827,  0.1618,  0.3560],
        [ 0.1773,  0.1129,  0.1030,  0.2155,  0.1681],
        [-0.3077, -0.1928, -0.3852, -0.1972, -0.0329],
        [-0.1731, -0.1554, -0.1834, -0.3334, -0.2544]],
       grad_fn=<SelectBackward0>)

Filter 8:
Shape: torch.Size([5, 5])
tensor([[-0.1110,  0.1264, -0.2479,  0.0717, -0.0176],
        [-0.2453, -0.0086, -0.0185,  0.2251,  0.2872],
        [ 0.1872,  0.0923,  0.0481,  0.0401, -0.0181],
        [-0.0368,  0.0644,  0.1833,  0.1320, -0.1457],
        [ 0.0450,  0.1382, -0.1793, -0.1820, -0.1543]],
       grad_fn=<SelectBackward0>)

Filter 9:
Shape: torch.Size([5, 5])
tensor([[-0.0904, -0.1695, -0.2104, -0.2565, -0.0798],
        [ 0.2653, -0.0182, -0.2360, -0.1935, -0.2251],
        [ 0.3296,  0.0225, -0.1420,  0.0075, -0.2216],
        [ 0.1802,  0.4438,  0.4609,  0.3932,  0.2794],
        [ 0.1163, -0.0455,  0.2375,  0.1528,  0.2643]],
       grad_fn=<SelectBackward0>)

Filter 10:
Shape: torch.Size([5, 5])
tensor([[ 0.2861,  0.3220,  0.2709,  0.1156,  0.0831],
        [-0.0015,  0.2573,  0.2778, -0.1133,  0.1931],
        [-0.0986,  0.1649,  0.0609, -0.0580,  0.2350],
        [ 0.1298, -0.0906, -0.0552,  0.1859, -0.0621],
        [-0.1433,  0.0234, -0.0178, -0.0960, -0.0591]],
       grad_fn=<SelectBackward0>)
```



Part B:

Overall, the results make sense as the filters have learned to detect different local features like edges, gradients, and stroke orientations rather than being random. No two filters seemed to have produced identical effects which confirms that the network learned a diver set of complementary feature detectors during training.

Filters 1 and 2 have a dark to light gradient diagonally across the filter where they seem to be detecting the diagonal edges of the digit.
Filters 5, 7, and 9 have a darker to light gradient across the horizontal axis which means these are most likely detecting the horizonal or vertical edges of the digit.



Task 3: Transfer Learning on Greek Letters

To adapt the MNIST network for Greek letter classification, we loaded the pretrained model and froze all layer weights to preserve the learned features. We then replaced the final fully connected layer (fc2) from a 50 to 10 mapping to a 50 to 3 mapping, corresponding to alpha, beta, and gamma. Only this new layer was trained using SGD with a learning rate of 0.05 and momentum of 0.9.

The Greek letter images were transformed to match MNIST format by converting to grayscale, scaling down, center cropping to 28x28, and inverting the intensities so the letters appear white on black. The same MNIST normalization (mean=0.1307, std=0.3081) was applied.

The model reached 100% training accuracy in just 2 epochs, showing that the pretrained convolutional layers already captured useful low-level features like edges and curves that transfer well to similar handwriting recognition tasks.

We then tested the model on 12 custom handwritten Greek letters drawn in Paint (4 per class). The model correctly classified 8 out of 12 (66.7%). Alpha and gamma were recognized perfectly at 4/4 each, while beta was misclassified in all 4 cases. This is likely because the training set only had 9 examples per class, giving the model limited exposure to variations in how beta can be written.

```
MyNetwork(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1))
  (pool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2): Conv2d(10, 20, kernel_size=(5, 5), stride=(1, 1))
  (dropout): Dropout(p=0.5, inplace=False)
  (pool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (fc1): Linear(in_features=320, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=3, bias=True)
)
```

Figure 3.1: Modified network for Greek Letters

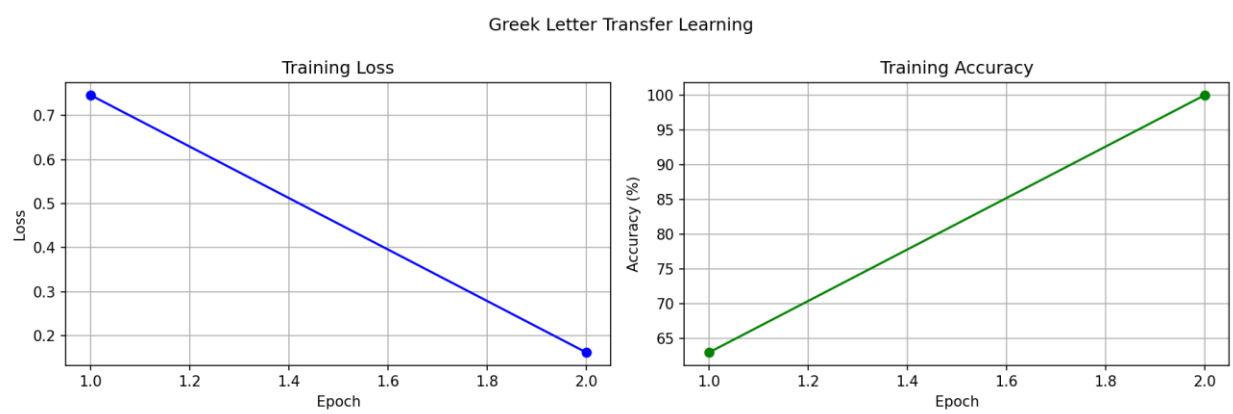


Figure 3.2: Training curves for Greek Letters

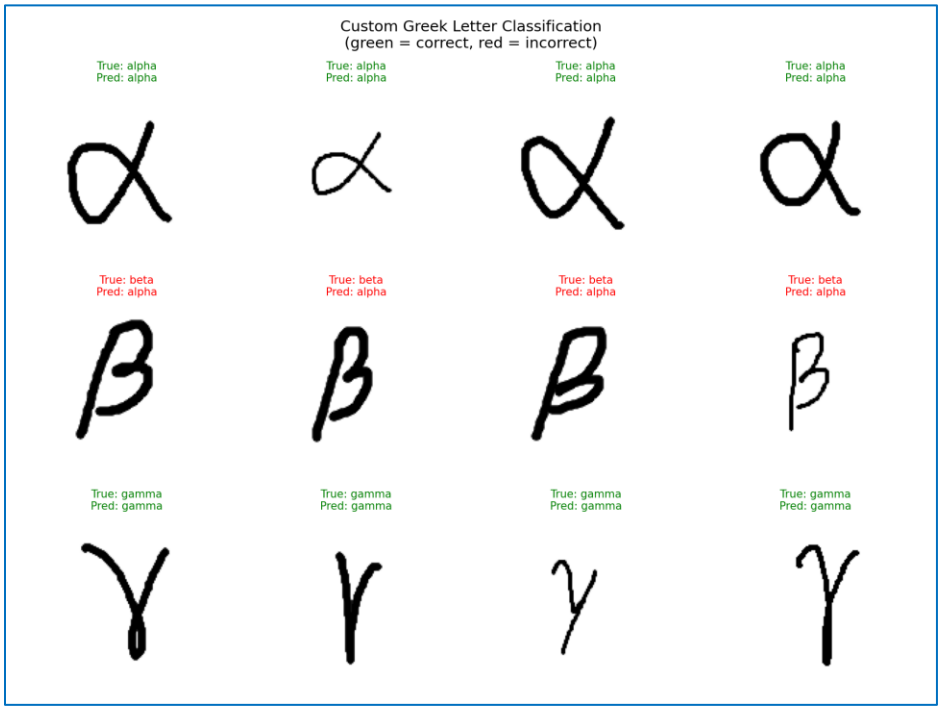


Figure 3.2: Testing results on the additional custom data

Task 4: Re-implement the network using transformer layers

We replaced the CNN architecture with a Vision Transformer using the provided template with default settings: patch size 4, stride 2, embedding dimension 48, 4 transformer layers, 8 attention heads, and MLP dimension 128. Instead of sliding convolutional filters, the transformer divides the 28x28 image into overlapping 4x4 patches (169 total), projects each patch into a 48-dimensional embedding, adds learned positional encodings, and processes the sequence through self-attention layers. The final classification averages all token representations and passes them through a linear classification head. The model was trained for 15 epochs using AdamW (lr=0.001, weight decay=0.0001) and reached approximately 99% test accuracy, comparable to the CNN.

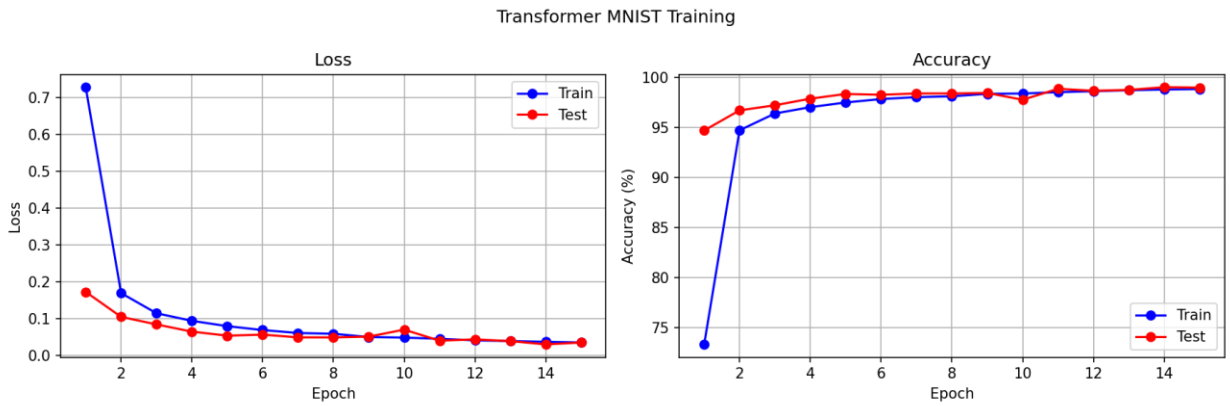


Figure 4.1: Training curves

The transformer had 104,842 parameters, roughly 5x more than the CNN, and took longer to train. Despite this, both architectures performed similarly on MNIST, which suggests the dataset is simple enough that the transformer's ability to capture long-range dependencies between patches does not provide a major advantage over local convolution filters.

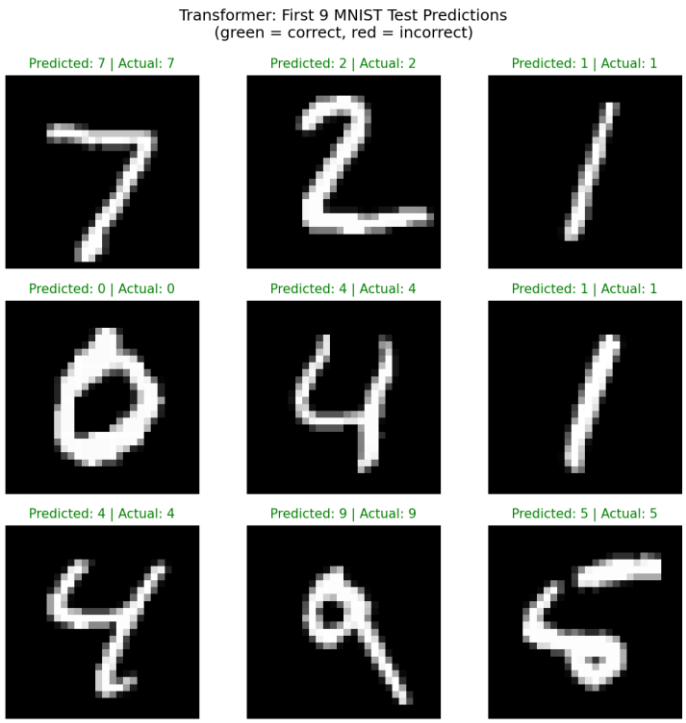


Figure 4.2: Transformer test predictions plot

Task 5: Design your own experiment

For task 5, we decided to experiment with the number of filters in the first convolution layer, number of hidden nodes in the fully connected layer, and the dropout rate to see how these dimensions impact how the model behaves on the MNIST fashion dataset.

Hypothesis for number of filters in the first convolution layer:
We believe that increasing the number of filters in the first convolution layer will improve accuracy up to a certain point since having more filters would allow the network to learn a better set of low-level feature detectors. After a certain point of number of filters, we believe that the improvements will begin to plateau or even decline due to overfitting.

Hypothesis for number of hidden nodes in the fully connected layer:
We believe that increasing the number of hidden nodes will improve accuracy as the larger dense layer gives the network more capacity to combine the convolutional features into class-level decisions. Since the fashion MNIST dataset is more complex than digit MNIST, the classifier head should benefit from this extra capacity. Again, once the number of hidden nodes reaches a certain point, we expect to see diminishing returns due to the risk of overfitting.

Hypothesis for dropout rate:
We believe that testing with too low of a dropout rate would provide insufficient regularization and may lead to overfitting on the fashion MNIST and too high of a dropout rate would zero out

activations too aggressively, reducing the network signal significantly and causing trouble for the network to converge.

Evaluation for number of filters in first convolution layer:

The results show steady improvement and peaks around 256 number of filters as it being to plateau and decline slightly which is what we expected as we would get diminishing returns for our results as we increased the number of filters. The runtime also increased as we increased the number of filters, which is a waste of resources for unreasonable jumps in performance.

Evaluation for number of hidden nodes in fully connected layer:
The difference between 50 hidden nodes and 1000 hidden nodes for test accuracy was less than 1%. Although the accuracy did increase as we increased the number of hidden layers, it seems like it's not worth the resources to such minuscule improvements.

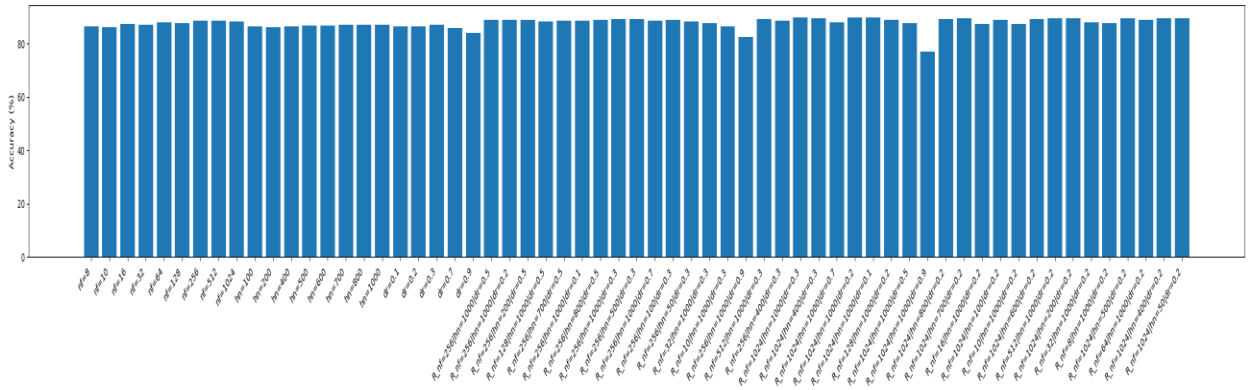
Evaluation for dropout rate:
The results agree with what we mentioned before and show that performance peaks at 0.3 dropout rate and drops noticeably at 0.7 and especially at 0.9, dropping more than 4% in test accuracy from the peak (0.3 dropout rate). This is a sign that the network is being over-regularized rather than getting better at generalizing.

Top Random Search Results:

Across the 39 competitve configurations, we examined the configurations with a dropoff threshold of 2.5% from the max accuracy from the resulting csv file generated from our experiment. The results seem to tell us that the number of filters seem to matter a lot as it was able to achieve the highest test accuracies when using more filters in the first convolutional layer. More hidden nodes also contribute to higher test accuracy, which is expected, and the sweet spot for dropout rate was around 0.2 - 0.3.

When speaking about the most optimal, it would be based on how you want to look at it. If you are looking for the best performance, the configuration that gave us the best performance was using 1024 number of filters, 1000 hidden nodes, and dropout rate of 0.2. If you are looking for the best overall result considering training time, then the configuration with 512 number of filters, 1000 hidden nodes, and dropout rate of 0.2 would be best. While that does sacrifice test accuracy SLIGHTLY, the training time was less than double what the peak test accuracy configuration was (275.9 seconds vs 118.7 seconds).

```
Max acc: 89.89%, Lowest acc: 87.39%
Competitive configs (39):
  R_nf=32|hn=1000|dr=0.3                    acc=87.58%  time=64.1s  params=347862
  nf=128                                    acc=87.60%  time=70.1s  params=83908
  R_nf=8|hn=1000|dr=0.2                     acc=87.78%  time=64.4s  params=335238
  R_nf=1024|hn=1000|dr=0.5                  acc=87.80%  time=173.8s  params=869654
  R_nf=32|hn=1000|dr=0.2                    acc=87.98%  time=64.5s  params=347862
  nf=64                                     acc=87.99%  time=62.5s  params=50244
  R_nf=1024|hn=1000|dr=0.7                  acc=88.05%  time=273.6s  params=869654
  R_nf=256|hn=50|dr=0.3                     acc=88.21%  time=112.4s  params=151236
  nf=1024                                   acc=88.33%  time=295.1s  params=555204
  R_nf=128|hn=1000|dr=0.5                   acc=88.35%  time=90.6s  params=398358
  nf=512                                    acc=88.56%  time=338.8s  params=285892
  R_nf=256|hn=1000|dr=0.7                   acc=88.56%  time=109.3s  params=465686
  nf=256                                    acc=88.62%  time=146.3s  params=151236
  R_nf=256|hn=400|dr=0.3                    acc=88.65%  time=101.7s  params=267086
  R_nf=256|hn=700|dr=0.5                    acc=88.72%  time=116.2s  params=366386
  R_nf=256|hn=1000|dr=0.1                   acc=88.72%  time=89.6s  params=465686
  R_nf=256|hn=800|dr=0.5                    acc=88.80%  time=94.1s  params=399486
  R_nf=256|hn=200|dr=0.5                    acc=88.85%  time=114.0s  params=200886
  R_nf=128|hn=1000|dr=0.2                   acc=88.87%  time=90.2s  params=398358
  R_nf=1024|hn=100|dr=0.2                   acc=88.88%  time=253.5s  params=571754
  R_nf=64|hn=1000|dr=0.2                    acc=88.88%  time=69.1s  params=364694
  R_nf=256|hn=1000|dr=0.2                   acc=88.95%  time=114.7s  params=465686
  R_nf=256|hn=100|dr=0.3                    acc=88.96%  time=112.1s  params=167786
  R_nf=256|hn=1000|dr=0.5                   acc=89.01%  time=110.3s  params=465686
  R_nf=256|hn=500|dr=0.3                    acc=89.09%  time=109.0s  params=300186
  R_nf=1024|hn=800|dr=0.2                   acc=89.18%  time=335.2s  params=803454
  R_nf=512|hn=1000|dr=0.3                   acc=89.19%  time=145.9s  params=600342
  R_nf=1024|hn=600|dr=0.2                   acc=89.25%  time=243.9s  params=737254
  R_nf=256|hn=1000|dr=0.3                   acc=89.35%  time=112.8s  params=465686
  R_nf=1024|hn=50|dr=0.2                    acc=89.44%  time=258.1s  params=555204
  R_nf=1024|hn=400|dr=0.2                   acc=89.46%  time=250.7s  params=671054
  R_nf=1024|hn=400|dr=0.3                   acc=89.47%  time=262.3s  params=671054
  R_nf=1024|hn=200|dr=0.2                   acc=89.49%  time=240.2s  params=604854
  R_nf=1024|hn=500|dr=0.2                   acc=89.49%  time=261.9s  params=704154
  R_nf=1024|hn=700|dr=0.2                   acc=89.51%  time=246.0s  params=770354
  R_nf=512|hn=1000|dr=0.2                   acc=89.56%  time=118.7s  params=600342
  R_nf=1024|hn=1000|dr=0.3                  acc=89.66%  time=184.3s  params=869654
  R_nf=1024|hn=1000|dr=0.1                  acc=89.77%  time=265.0s  params=869654
  R_nf=1024|hn=1000|dr=0.2                  acc=89.89%  time=275.9s  params=869654
```



Extensions:

1. Transfer Learning with Additional Greek Letters

As an extension, we expanded the Greek letter classification from 3 classes (alpha, beta, gamma) to 6 classes by adding delta, theta, and lambda. Each new class had 9 training examples to match the original dataset, and 4 custom handwritten test images were drawn per class.

With only fc2 trainable (as in the base task), the model reached 100% training accuracy but performed poorly on custom test images. The frozen 50-dimensional features from fc1 were optimized for digit classification and could not produce separable representations for 6 Greek letter classes through a single linear layer.

To address this, we unfroze fc1 as well, allowing the network to reshape its 50 intermediate features while keeping the convolutional layers frozen for low-level edge and curve detection. This improved **test accuracy to 14/24 (58.3%)**. Delta and theta were classified perfectly at 4/4 each, while alpha and beta were the weakest performers due to their more complex and variable shapes.

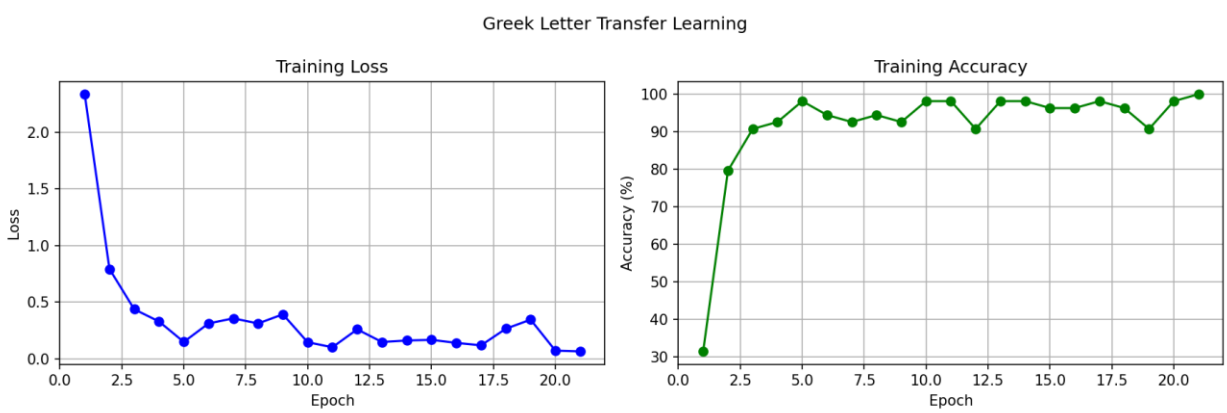


Figure: 6-class training curves

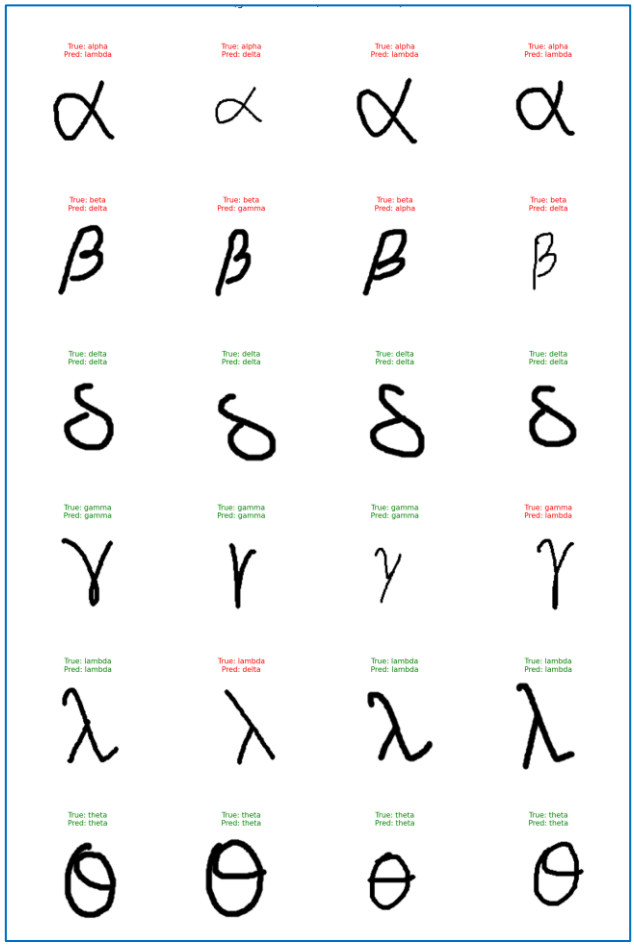


Figure: 6-class test results

2. Interactive Digit Recognition with Gradio

We built an interactive web interface using Gradio that lets users draw a digit on a sketchpad and get real-time predictions from the trained MNIST model. The drawn image is converted to grayscale, resized to 28x28, inverted to match MNIST's white-on-black format, and normalized before being passed through the network. The interface displays the top 3 predicted classes with confidence scores.

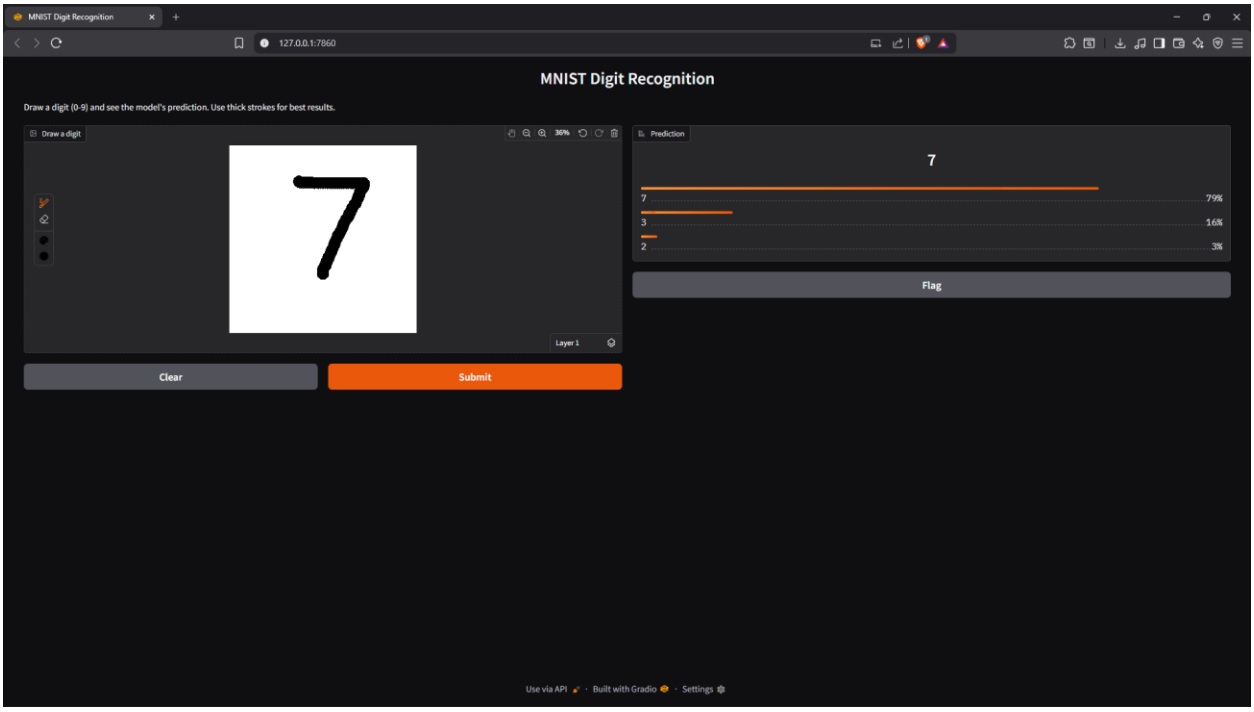


Figure: Sketch pad interface for Digit Recognition

Reflection:

This project gave us a solid understanding of deep learning for image recognition, from building a CNN from scratch to exploring modern transformer architectures. Training on MNIST showed how convolutional layers learn increasingly complex features, from simple edges in early layers to digit-level patterns in deeper ones. Transfer learning demonstrated that these learned features are reusable, classifying Greek letters in just 2 epochs, though expanding to 6 classes exposed the limits of frozen features and required unfreezing additional layers. Implementing the Vision Transformer highlighted how different architectures can solve the same problem, with patches and self-attention replacing convolutions yet achieving comparable results. The experiment task reinforced that network design involves balancing many interacting parameters and that systematic evaluation is key to understanding their effects.

Acknowledgements:

- PyTorch documentation and tutorials for CNN and transformer implementation
- Course materials and project template provided by Professor Bruce Maxwell and Andy Zhao
- Claude AI for debugging assistance